

ANTLAB

ProofFlow AI Technical Specification

Architecture, smart contract integration, LLM pipeline design, and deployment roadmap for an AI-powered Web3 pre-launch verification platform

Prepared by AntLab | Web3 Technical Documentation | Confidential

Executive Summary

ProofFlow AI is a pre-launch verification platform that automatically cross-references Web3 project whitepapers, GitHub repositories, contract ABIs, and tokenomics data to identify and flag potential documentation gaps, technical contradictions, and investor-risk signals for specialist review. The platform produces a structured, severity-tiered risk report for founders, auditors, launchpads, and venture analysts - a pre-audit documentation screening layer that complements, not replaces, manual security audits (which cost \$20,000-\$500,000 per engagement).

- MVP scope: whitepaper PDF, GitHub repo, contract ABI, and token distribution ingestion on Ethereum and EVM-compatible L2s (website scraping deferred to Phase 2)
- Core output: structured PDF and web dashboard risk report with severity-tiered findings across 5 check categories (tokenomics vs contract, team vs GitHub, security vs ABI, roadmap vs code, supply vs on-chain)
- Primary LLM: GPT-4o or Claude 3.5 Sonnet with a deterministic rule layer for high-confidence flags
- Team: 5 specialists (2 full-stack, 1 AI/LLM, 1 smart contract security, 1 product designer)
- Timeline: 12-week baseline MVP (10-week min / 14-week contingency); Budget: \$40,000-\$70,000
- Monetization: per-report pricing (\$200-\$2,000 by complexity) + API subscription tier for launchpad partners

MARKET CONTEXT

The smart contract audit market reached \$890M in 2024 and is projected to grow to \$6.1B by 2033 (22.8% CAGR). ProofFlow targets an estimated \$90M-\$130M pre-audit documentation screening sub-segment - an analyst estimate based on 10-15% of total audit TAM. No direct market data exists for this specific niche; this figure should not be cited as a sourced market statistic.

KEY DIFFERENTIATOR AND COMPETITIVE POSITION

As of mid-2025, no existing competitor (CertiK Skynet, Token Sniffer, GoPlus, De.Fi) ingests and cross-validates pre-launch documentation artifacts against contract ABIs as a holistic system. This niche is not permanently protected: well-funded incumbents could replicate the documentation ingestion layer within 3-6 months. Defensibility relies on proprietary labeled benchmark data, launchpad integration switching costs, and specialist annotation creating domain-specific training data. Competitive claim valid as of document date; refresh before each external distribution.

Problem Statement

Web3 projects routinely launch with incomplete, inconsistent, or misleading technical documentation. Launchpads, auditors, and investors must manually reconcile whitepapers, contract code, tokenomics tables, and website claims - a slow, expensive, error-prone process with no standardized automated pre-audit layer. With approximately 2,000 blockchain security specialists globally and \$3.35B lost across 630 security incidents in 2025 (CertiK Hack3D), the cost of inadequate pre-launch due diligence is measurable and severe.

- Auditors bill \$20,000-\$500,000 per engagement; early-stage projects cannot afford full audits before documentation is validated
- All existing scanners operate exclusively on deployed contracts - none ingest or cross-validate pre-launch documentation artifacts (as of mid-2025)
- Launchpads and VCs reviewing 100+ project applications per cohort lack a scalable, programmatic screening layer
- LLM-based cross-referencing of multi-source documents is a novel capability not offered by CertiK, GoPlus, or Token Sniffer at the time of writing
- Founders receive no structured, actionable pre-audit feedback before committing to expensive formal reviews

CORE GAP

There is no tool that correlates off-chain documentation artifacts (whitepapers, GitHub READMEs, tokenomics tables) against on-chain contract ABIs and token distribution data to detect contradictions before launch. ProofFlow closes this gap with a deterministic-plus-LLM cross-reference engine covering 5 check categories: (1) tokenomics vs contract, (2) team vs GitHub, (3) security vs ABI, (4) roadmap vs code status, (5) supply distribution vs on-chain data.

Solution Architecture

ProofFlow's MVP architecture is a four-stage linear pipeline: Ingestion, Parsing, Cross-Reference, and Report Generation. The system is entirely read-only - no blockchain write operations - making it safe to deploy serverlessly within the \$40,000-\$70,000 budget envelope. Backend is Python/FastAPI with Celery and Redis for async job queuing; frontend is Next.js; data store is PostgreSQL. Website scraping via Playwright/Puppeteer is explicitly deferred to Phase 2; the MVP ingestion layer accepts only user-submitted content.

- Ingestion (MVP): whitepaper PDF URL/upload, GitHub repo URL, EVM contract address, token distribution CSV - website URL deferred to Phase 2
- Parsing (MVP): pypdf or pdfplumber for PDFs (PyPDF2 is deprecated; use pypdf); GitHub REST API for repos; Ethers.js/Viem + Etherscan API for ABIs
- Cross-Reference Engine: LangChain/LlamaIndex RAG + GPT-4o structured output (function calling) + Slither static analysis + deterministic rule layer across all 5 check categories
- Report Generator: Pydantic-validated JSON rendered as PDF and Next.js dashboard with severity tiers (Critical, High, Medium, Informational); finding objects carry id, category, severity, citations, confidence, remediation, and specialist_override
- Infrastructure: AWS Lambda or Vercel Edge for bursty ingestion; Redis for queue; S3 for artifact storage

ARCHITECTURE DECISION

A hybrid deterministic-plus-LLM engine is mandatory for trust. Pure LLM output without a rule layer creates hallucination risk that would destroy credibility with auditors and launchpads. Deterministic rules handle boolean checks (ABI function presence, supply math); the LLM handles semantic inconsistency detection across prose. All LLM calls must use structured output mode (function calling / tool use) to constrain format and reduce prompt injection surface.

Smart Contract Integration

ProofFlow is a read-only analysis platform with no on-chain deployment in MVP scope. The smart contract integration layer is a set of off-chain ABI parsing, static analysis, and token distribution verification modules. Slither (Trail of Bits) performs static analysis on verified Solidity source; Ethers.js/Viem handles ABI fetching and on-chain supply reads from Etherscan, Basescan, Arbiscan, and Polygonscan. Unverified contracts return only bytecode and are flagged as a Critical finding.

Analysis Module	Method	Input	Output	Chains
ABI Fetcher	Etherscan API + Viem	Contract address	ABI JSON + verified flag	ETH, Base, Arb, Polygon
Static Analyzer	Slither CLI	Verified Solidity source	Vulnerability list + function graph	EVM (all)
Token Supply Reader	Ethers.js totalSupply()	ERC-20 address	On-chain supply figure	EVM (all)
Ownership Checker	Ethers.js owner()/admin()	Contract address	Privileged role addresses	EVM (all)
Distribution Verifier	On-chain balance reads	Wallet list from CSV	Claimed vs actual allocation delta	EVM (all)

CRITICAL RULE

If a contract address is provided but Etherscan returns unverified bytecode only, ProofFlow must immediately surface a Critical severity flag: 'Contract source unverified - static analysis impossible; formal audit required before launch.' This addresses check category (3) security claims vs ABI and prevents false negatives from unverified contracts silently passing checks.

Backend and API Specification

The backend is a Python FastAPI application exposing a RESTful API for job submission, status polling, and report retrieval. Celery workers with a Redis broker handle async multi-source ingestion; each submission spawns parallel ingestion tasks per source type, converging into the cross-reference engine only after all sources resolve or timeout. PostgreSQL stores structured report JSON, job state, and org records. The job state machine transitions: PENDING -> INGESTING -> PARSING -> CROSS_REFERENCING -> GENERATING -> REVIEW_PENDING -> COMPLETE (or FAILED at any stage).

```
# Core API endpoints (FastAPI)
POST /api/v1/projects # Submit project for analysis
GET /api/v1/projects/{id} # Poll job status
GET /api/v1/reports/{id} # Fetch structured JSON report
GET /api/v1/reports/{id}/pdf # Download PDF report
PATCH /api/v1/reports/{id}/annotations # Specialist manual override
GET /api/v1/health # Service health check

# Finding object schema (Pydantic)
{
  "id": "uuid",
  "category": "tokenomics|team|security|roadmap|supply",
  "severity": "critical|high|medium|informational",
  "title": "string", "description": "string",
  "source_citations": ["string"],
  "confidence_level": 0.0, "is_deterministic": true,
  "remediation": "string", "specialist_override": null
}
```

LLMCOSTCONTROL

GPT-4o API costs are projected at \$500-\$5,000/month at MVP volumes (analyst estimate based on public pricing, assuming 50-200 analyses/month at ~20,000-60,000 tokens per job). Implement token budgeting per job: chunk large PDFs into 4,000-token segments with LlamaIndex, cache parsed document embeddings (Pinecone or pgvector), and set a hard per-job LLM call limit to prevent runaway costs during beta.

Frontend Integration

The frontend is a Next.js 14 (App Router) dashboard that renders structured report JSON into a launchpad-grade interface. Authentication uses Web3 wallet connection (wagmi + RainbowKit - confirm current compatible versions, as wagmi v1 and v2 have breaking API differences) or email/password for non-crypto users. The report view presents severity-tiered findings in collapsible cards, a risk score summary, a source-citation panel linking each finding to its originating document, and a PDF export action.

- Project submission form: URL/file inputs for whitepaper, GitHub, contract address, tokenomics CSV; website URL input shown but disabled, labeled 'Coming in Phase 2'
- Job status page: real-time polling with per-source ingestion progress reflecting the full state machine
- Report dashboard: Critical/High/Medium/Informational finding cards across all 5 check categories with citations and fix recommendations
- Specialist annotation panel: security specialist can add notes, override severity, or mark findings as false positives before delivery
- PDF export: server-rendered via Puppeteer or React-PDF; branded for launchpad white-label delivery

UX PRIORITY

The report must be interpretable by a non-technical VC analyst, not just a Solidity developer. Each finding requires a plain-English explanation, a severity badge, a source citation (e.g., 'Whitepaper section 4.2 vs ABI line 87'), and a remediation step. Raw JSON dumps are not acceptable. All reports display a legally reviewed disclaimer: 'ProofFlow findings are informational pre-audit signals, not a formal security audit or investment advice.' Final disclaimer language is subject to legal review before beta onboarding.

Security Considerations

ProofFlow's primary security surface is not on-chain but operational: the platform processes sensitive pre-launch project data (unpublished whitepapers, unreleased tokenomics) that must be stored and transmitted securely. All uploaded documents are encrypted at rest (AES-256 via AWS S3 SSE) and in transit (TLS 1.3). Report access is scoped to the submitting organization via JWT-authenticated API tokens. OpenAI Enterprise or Anthropic's zero-data-retention API tier must be used for all document processing.

- LLM hallucination mitigation: all Critical and High findings must be confirmed by at least one deterministic rule check; LLM-only findings are labeled 'AI-detected, unconfirmed' and visually separated
- Prompt injection threat model: adversarial instructions embedded in user-submitted documents are a distinct attack vector; mitigations include system prompt isolation, strict instruction/data separation, structured output mode, schema validation, and the specialist review gate as human-in-the-loop
- False-negative legal liability: an incorrect low-risk report followed by a project failure could expose ProofFlow to claims; the mandatory disclaimer is the primary contractual mitigation; legal scope must cover indemnification, liability caps, and the screening-vs-audit distinction
- Input sanitization: PDFs and GitHub content parsed in sandboxed workers; malicious files must not execute arbitrary code on the parsing host
- Data retention: raw documents deleted after 30 days; structured reports retained 12 months; content not used for LLM training; subprocessors include

- OpenAI/Anthropic (zero-retention tier) and AWS; GDPR/CCPA applicability to be confirmed with legal counsel

TRUST ARCHITECTURE

A deterministic rule layer is non-negotiable for production credibility. LLM-only findings must be labeled 'AI-detected, unconfirmed' and separated from rule-validated findings. This two-tier presentation preserves trust with auditor and launchpad audiences. No Critical-severity findings may be delivered to beta customers without confirmation by at least one deterministic rule check and specialist manual sign-off.

Testing Strategy

Testing spans unit tests on deterministic parsing modules, integration tests on the full ingestion-to-report pipeline, and a domain-specific LLM evaluation suite measuring false positive and false negative rates against curated real Web3 project documents. The LLM evaluation suite is the most critical and most novel workstream - without it, report quality cannot be quantified before beta. The benchmark dataset must include at minimum 30 labeled projects (known rugs, known exploits, clean audited projects, borderline cases) for beta baseline, expanding to 100+ for production.

- Unit tests (pytest): ABI parser, PDF extractor (pypdf/pdfplumber), tokenomics CSV validator, Slither output parser - target 90% coverage on deterministic modules
- Integration tests: end-to-end job submission through report generation on 10-15 real historical projects (including known rugs and clean projects)
- LLM evaluation: human-annotated ground-truth set of 30+ project document pairs (beta baseline); measure precision/recall per category using DeepEval or custom harness; 20-example sets are statistically insufficient across 5 categories
- Load testing: Celery worker throughput under concurrent 10-job bursts using Locust; ensure Redis queue does not bottleneck
- Beta QA: 3-5 launchpad or VC users review reports on live submissions; structured feedback captured per finding

Q A G A T E

No report may be delivered to a beta customer unless it has passed the specialist manual review gate. During MVP, the smart contract / security specialist reviews 100% of generated reports before delivery. Automating this gate is a post-MVP milestone gated on achieving measurable false-positive rates below a category-specific threshold (15% is a working placeholder pending baseline measurement, not a validated industry standard; final thresholds must be validated with at least one launchpad partner before automation).

Deployment Roadmap

The baseline MVP roadmap is 12 weeks across six phases with strict scope gates. The 10-week plan is minimum viable scope; two weeks of contingency (Weeks 11-12) are built in for LLM calibration and PDF parsing complexity. A further 14-week outer contingency exists if calibration proves harder than estimated. Website scraping, multi-chain expansion beyond Ethereum, third-party API access, and automated leaderboards are explicitly deferred to post-MVP. The Weeks 3-4 parsing spike is a hard gate: LLM integration does not begin until the parser gate criterion is met.

Phase	Weeks	Deliverables	Gate Criteria
Architecture & Ingestion	1-2	FastAPI skeleton, Celery/Redis queue, PDF parser, ABI fetcher, GitHub ingestion (website URL excluded)	All three MVP source types (PDF, GitHub, ABI) return structured JSON for a real project
Parsing Robustness Spike	3-4	PDF parser hardened across formats; GitHub README + CSV extraction validated; Slither tested on verified contracts	Clean structured JSON for ≥ 10 diverse whitepapers with no critical extraction failures; signed off before Week 5
Cross-Reference Engine	5-7	LLM pipeline on validated parser output; deterministic rules for all 5 categories; GPT-4o function calling; injection mitigations	Engine flags ≥ 3 known inconsistencies across at least 3 of 5 check categories

Phase	Weeks	Deliverables	Gate Criteria
Report Generator & Dashboard	7-9	Pydantic schema, PDF export, Next.js dashboard, severity tiers, annotation panel, disclaimer	Full report for 5 real projects; PDF passes launchpad readability; disclaimer approved by legal
QA & Beta Onboarding	10-12	30+ project eval baseline, false-positive calibration, 3-5 beta users, legal copy finalized	Beta users submit ≥ 3 real projects; no Critical findings delivered without rule confirmation + specialist sign-off
Post-MVP (Week 13+)	13+	Website scraping (Playwright), Base/Arbitrum/Polygon support, API subscription tier, scoring leaderboard	Dependent on beta feedback, PMF signals, and revenue runway

SCOPEDISCIPLINE

The single highest timeline risk is PDF parsing quality and LLM prompt calibration across diverse whitepaper formats. Weeks 3-4 are dedicated exclusively to a parsing robustness spike before LLM integration begins in Week 5. Do not parallelize parser development with LLM integration - bad parser output propagates errors through the entire pipeline. Website scraping via Playwright is deferred to Phase 2 (Week 13+) and must not appear in any MVP sprint.

Cost Estimation

The \$40,000-\$70,000 budget implies approximately \$8,000-\$14,000 per team member over 12 weeks, workable only if the team is equity-compensated or engaged as contractors at below-market rates. Infrastructure and LLM API costs are variable but manageable at MVP volumes (50-200 analyses/month) with strict token budgeting. The largest budget risk is LLM API overage during testing without per-job cost caps enforced in code. Engage legal before beta onboarding, not after.

Workstream	Low (USD)	High (USD)	Notes
Team Compensation (12 wks, 5 people)	\$38,000	\$60,000	Contractor or equity-adjusted rates; not market salaries; 12-week baseline adds ~\$5-6K vs 10-week
LLM API Costs (MVP testing)	\$1,500	\$6,000	50-200 analyses/month at ~20-60k tokens/job; enforce per-job token caps; verify against current pricing
Cloud Infrastructure (AWS/Vercel, Redis, RDS, S3)	\$800	\$2,500	Serverless + managed DB; scales with job volume
Third-Party APIs (Etherscan, GitHub Pro, Pinecone)	\$300	\$800	Etherscan Pro ~\$199/mo; GitHub API free tier sufficient for MVP
Legal / Disclaimer Review	\$500	\$2,000	One-time US tech/fintech attorney; covers disclaimer, liability caps, indemnification, false-negative scenario
QA, Tooling, and Contingency	\$900	\$3,700	Eval harness tooling, load-test credits, 10% contingency buffer

BUDGET WARNING

At \$40,000, the budget is viable only with equity compensation and zero scope creep. At \$70,000, there is sufficient runway for the full 12-week MVP plus one month of post-launch iteration. If LLM calibration extends the timeline to 14 weeks (outer contingency), an additional \$10,000-\$15,000 in team costs should be anticipated. Business model context: at \$200-\$2,000 per report and 50-200 analyses/month, monthly gross revenue at MVP scale ranges \$10,000-\$400,000 before LLM and infra costs - making unit economics viable if beta conversion to paid is achieved.